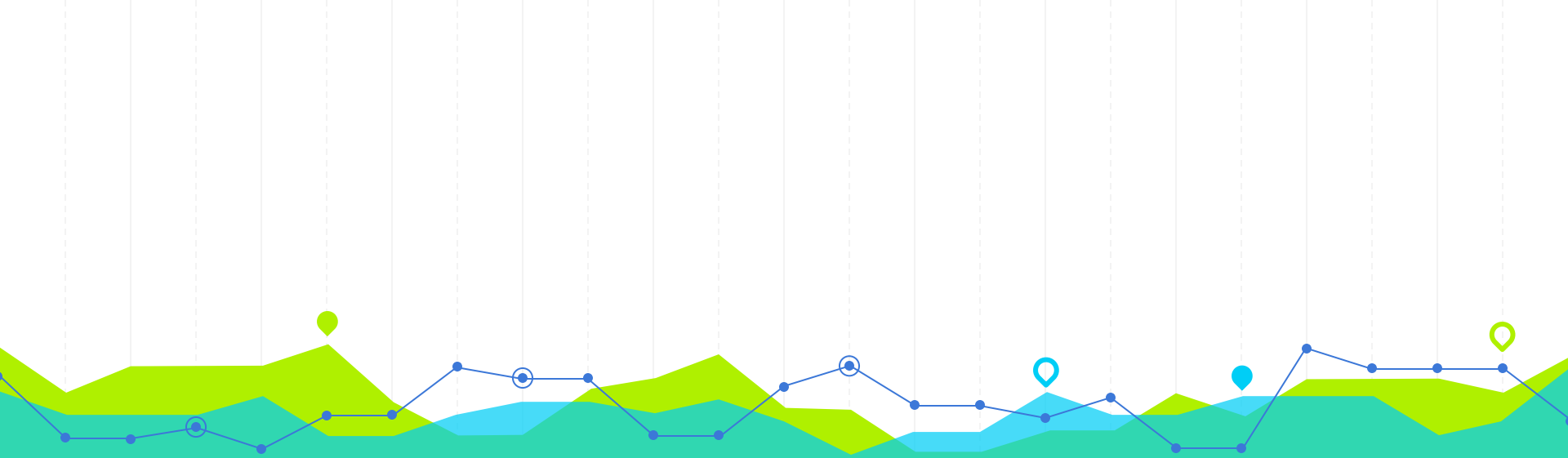




Coder en Python

pour la production

Le programme - BGD700

	Lambert FATOUX		Charlotte LACLAU	Charlotte LACLAU Lambert FATOUX	... Deadline le 1 Décembre à 23h59 Envoyer le lien GitHub du projet à lambert.fatoux@enpc.fr N'oubliez pas de préciser dans le mail les noms des membres de votre équipe
8:30 - 10:00	Présentations	Coder avec les bonnes pratiques	Analyse de données	QCM sans internet ni supports (10 pts)	
	Gérer son environnement de développement				
10:00 - 10:15					
10:15 - 11h:45				Journée sur le projet	
13:30 - 15:00	Créer et coder dans un projet Python	L'intégration Continue & Développement Continu			
15:00 - 15:15					
15:15 - 16:45		Webapp avec Streamlit			
		Partage du projet (10 pts) à faire			
à la maison		Projet			

Deadline le **1 Décembre à 23h59**
Envoyer le lien GitHub du projet à
lambert.fatoux@enpc.fr

N'oubliez pas de préciser dans le mail les noms des membres de votre équipe

Pourquoi est-il important d'écrire du code de qualité en production ?

- Mais ça marche sur mon poste...
- OK on l'envoie au client

Moi :

« Être dev, c'est pas produire un code qui marche »
« Ça, n'importe quel dev avec 2-3 mois de métier peut le faire »
« Non, notre métier c'est de produire un code durable. »



Dev:
Ca marche sur
mon pc

PM:
Oui mais on va
pas donner ton
pc au client



© 2019 Compagnon.

Pourquoi est-il important d'écrire du code de qualité en production

1. **fiabilité** : réduit le risque de **bugs** inattendus en production => **maintient la confiance** des utilisateurs
2. **maintenance** : un code bien structuré et propre est plus facile à **maintenir**
3. **scalabilité** : ne pas tout refaire si l'on veut pouvoir passer à l'échelle
4. **collaboration** : une équipe **comprendra mieux** un code bien organisé, documenté et respectant les conventions
5. **performance** : un code **sans redondances** et avec algorithmes **optimisés** est **plus performant**

Pourquoi est-il important d'écrire du code de qualité en production ?

6. **sécurité** : un code de mauvaise qualité peut présenter des **failles** de sécurité
7. **durée de vie du produit** : un code de qualité assure que le logiciel peut **évoluer** et **s'adapter aux besoins**.
8. **économies de coûts** : écrire du code de qualité prend plus de temps, mais entraîne d'importantes **économies à long terme**
9. **satisfaction professionnelle** (et ça augmente la **confiance** que les autres ont en votre travail 😊)

**Le code de qualité n'est pas seulement une question d'esthétique ou de préférence personnelle.
Il s'agit d'une nécessité pour assurer la fiabilité, la sécurité, la performance et la pérennité des systèmes en production.**

Le programme

Sur 12 heures

1. Gérer son environnement de développement
2. Créer et coder un projet Python
3. Appliquer les bonnes pratiques
4. L'intégration Continue & Développement Continu

		J1	J2
8:30 - 10:00		Présentations	Coder avec les bonnes pratiques
		Gérer son environnement de développement	
10:00 - 10:15			
10:15 - 11h:45			
13:30 - 15:00		Créer et coder dans un projet Python	L'intégration Continue & Développement Continu
15:00 - 15:15			
15:15 - 16:45			Webapp avec Streamlit

Le programme

Gérer son environnement de développement

Objectifs

- prendre en main un IDE pour une utilisation professionnelle
- gérer correctement ses environnements Python

Programme

1. **Choisir** son IDE
2. **Configurer** son IDE
3. Choisir sa **version** de Python
4. Créer ses **environnements** Python
5. Choisir ses **librairies** Python
6. Utiliser le **debugger** de son IDE

	J1 matin
	Présentations
8:30 - 10:00	Gérer son environnement de développement
10:00 - 10:15	
10:15 - 11h:45	

Le programme

Créer et coder un projet Python

Objectifs

- comprendre comment est structuré un projet python
- comprendre la Programmation Orientée Objet en python
- savoir utiliser Git en entreprise

Programme

1. Structure d'un **projet Python**
2. POO en Python
3. Savoir utiliser **Git en CLI**
4. Savoir utiliser les **fonctionnalités** de GitHub / GitLab

	J1 après-midi
13:30 - 15:00	Créer et coder dans un projet Python
15:00 - 15:15	
15:15 - 16:45	

	J2 matin
8:30 - 10:00	Coder avec les bonnes pratiques
10:00 - 10:15	
10:15 - 11h:45	

Le programme

Coder avec les bonnes pratiques

Objectifs

- écrire du code propre, performant, robuste et de qualité

Programme

1. Introduction
2. Code **clean** avec les guidelines **pep8**
3. **Logger** son application
4. **Sécuriser** son application
5. **Documenter** son code
6. **Optimiser** son code
7. **Tester** son code

	J2 après-midi
13:30 - 15:00	CI / CD Intégration Continue & Développement Continu
15:00 - 15:15	
15:15 - 16:45	Webapp avec Streamlit

Le programme

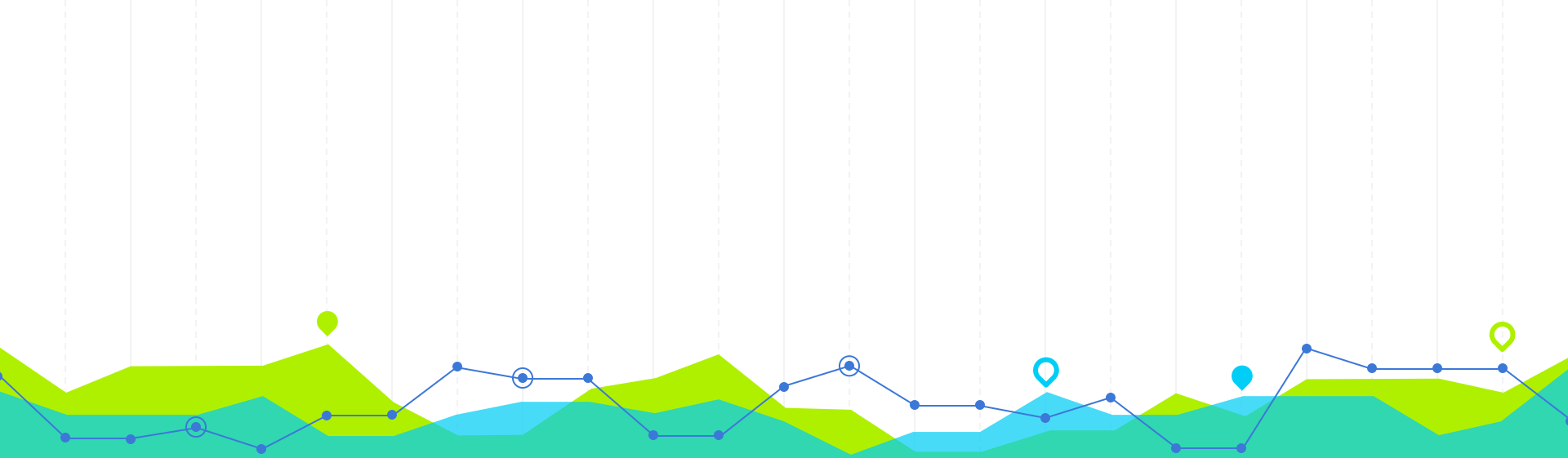
CI / CD et Streamlit

Objectifs

- Comprendre et construire un pipeline CI/CD
- Introduction aux webapps avec Streamlit

Programme

1. Introduction à la CI/CD
2. L'Intégration Continue (CI)
 - Plan
 - Code
 - Build un projet
 - Tester un projet
3. Le Développement Continu (CD)
 - Relancer un projet
 - Déployer un projet
 - Operate
 - Monitoring
4. Webapp avec Streamlit



Environnement de développement

Sommaire

1. **Choisir** son IDE
2. **Configurer** son IDE
3. Choisir sa **version** de Python
4. Créer ses **environnements** Python
5. Choisir ses **librairies** Python
6. Utiliser le **debugger** de son IDE

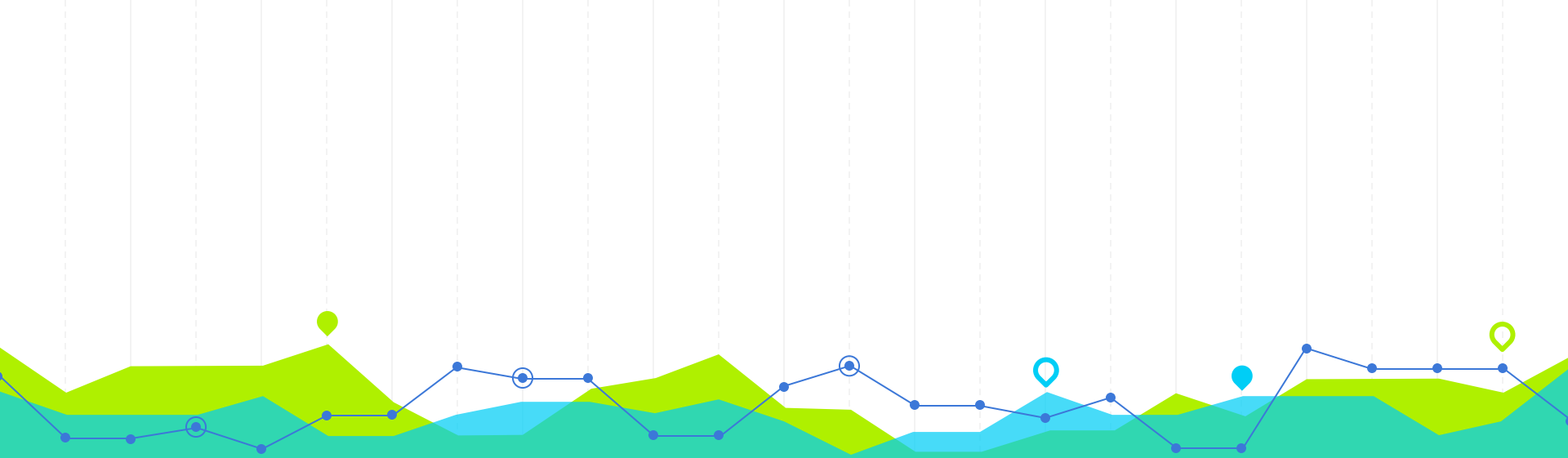


Objectif

Construire son environnement de
développement

Un bon environnement de dev
= plus agréable et plus productif !





1. Choisir son IDE

Qu'est ce qu'un IDE ?

Un IDE = Integrated **D**evelopment **E**nvironment (Environnement de Développement Intégré en français)

Outil logiciel qui fournit un ensemble de **fonctionnalités facilitant le développement** de programmes

Il offre des fonctionnalités spécifiques qui rendent le **codage**, le **test** et le **débogage** plus pratiques.

IDE = de nombreux avantages pour un développeur Python, surtout lorsque l'on compare à la programmation dans des **éditeurs de texte simples**



Pourquoi utiliser un IDE ?

- **Coder plus vite**
 - **Autocomplétion** : accélère la saisie du code, via la suggestion des noms de variables, de fonctions, etc.
 - **Documentation intégrée**: visualiser directement la documentation des bibliothèques, des classes, des méthodes, etc.
 - **Refactoring** : automatise des modifications structurelles du code sans changer son comportement.
- **Mieux déboguer**
 - **Analyse du code** : identifie les erreurs potentielles, les problèmes de style, ou d'autres problèmes avant même que le code ne soit exécuté.
 - **Débogage intégré** : met en pause l'exécution, place des points d'arrêt, inspecte les variables et permet de parcourir le code pas à pas.
- **Mieux intégré avec le reste**
 - **Intégration avec des outils tiers** : des outils de contrôle de version (comme Git), des BDDs, serveurs, outils de CI/CD, etc.
 - **Gestion des dépendances** : pip ou conda intégrés dans l'IDE pour gérer les packages Python et les environnements virtuels.
 - **Tests intégrés** : facilitent l'écriture, l'exécution et la surveillance des tests unitaires et fonctionnels.





Pourquoi utiliser un IDE ?

- **Support pour plusieurs langages** pour faciliter le passage d'un langage à un autre sur des **projets polyglottes**
- **Interface utilisateur riche** avec des menus, des boutons et des onglets, peut rendre certaines tâches plus intuitives et rapides par rapport à la ligne de commande (ex : Git)
- **Thèmes et personnalisation** pour adapter l'environnement selon les préférences de l'utilisateur
- **Éditeur de code** avec une coloration syntaxique, ce qui rend le code plus lisible
- **Intégration du compilateur ou de l'interpréteur** pour exécuter directement le code depuis l'environnement, sans avoir à utiliser une ligne de commande séparée
- **Gestion des projets** pour gérer des **projets entiers**
- **Explorateur de code** pour faciliter la navigation dans de grands projets, un explorateur de code peut vous montrer une vue structurée de tous vos fichiers, classes, méthodes, etc.



Les IDE existants pour Python

- **Spyder**
 - Idéal pour les **débutants**
 - Souvent comparé à MATLAB, Spyder est particulièrement apprécié des scientifiques et des ingénieurs.
 - Intégré avec **Anaconda**, une distribution populaire pour la data science.
- **Atom**
 - Éditeur de texte **gratuit** et **open source** développé par GitHub.
 - Avec le package **hydrogen**, il peut se comporter comme un environnement interactif similaire à Jupyter.
- **Thonny**
 - Idéal pour les **débutants**
 - Vient avec un débogueur intégré et facilite la gestion des environnements virtuels.
- **Wing IDE**
 - Dispose d'un puissant débogueur et est conçu pour être un **IDE professionnel** pour Python.
 - Propose une version gratuite et des versions payantes avec des fonctionnalités supplémentaires.
- **Eclipse avec PyDev**
 - Eclipse est un IDE pour **plusieurs langages**. Avec le plugin PyDev, il devient un IDE complet pour Python.
- **Komodo IDE**
 - Développé par ActiveState, Komodo offre des fonctionnalités pour **plusieurs langages**, dont Python.

Les IDE existants pour Python



Voici quelques-uns des meilleurs IDE pour Python :

- **IDLE**
 - L'IDE **standard** qui est fourni avec Python.
 - Plus simple que d'autres IDE, mais suffisant pour des scripts basiques.
- **Jupyter Notebook / JupyterLab**
 - Idéal pour la data science, l'analyse et la recherche. Mais **non adapté** pour du code de production.
 - Permet d'exécuter du code Python dans un format de **cellules interactives** avec des visualisations et du texte.
- **PyCharm**
 - Développé par JetBrains, c'est probablement l'un des IDE les plus **populaires** pour Python.
 - une version gratuite (Community) et une version payante (Professional) avec des fonctionnalités avancées.
 - Intégration avec de nombreux frameworks et bibliothèques, débogage, gestion des environnements virtuels, etc.
- **Visual Studio Code (VSCode)**
 - **Gratuit** développé par Microsoft
 - Extrêmement **modulable** avec une multitude d'**extensions**, dont l'extension Python qui est très puissante.
 - Prise en charge du débogage, linting, Jupyter notebooks, et bien d'autres fonctionnalités via des extensions
 - il existe une version **open source**, **VSCodium**, qui n'envoie pas de données à Microsoft

Le nouveau CEO d'AWS a dit récemment que bientôt, la majorité des développeurs ne coderont plus.

Matt Garman a récemment partagé sa vision : il prévoit que coder ne sera plus la compétence principale des développeurs dans un avenir proche.

Il dit même que « Être développeur en 2025 pourrait être bien différent de ce que cela signifiait en 2020 ».

Matt Garman explique que l'IA pourrait bientôt prendre en charge une grande partie du travail de coding, permettant ainsi aux développeurs de se concentrer davantage sur la compréhension des besoins des utilisateurs et la création de solutions à fort impact.

Il imagine un futur où l'IA gère les tâches les plus lourdes, et où les développeurs deviennent des architectes, des innovateurs, et des créateurs.

- Satya Nadella, CEO de Microsoft, anticipe que l'accès facilité aux technologies d'IA pourrait créer un milliard de développeurs

- Emad Mostaque, ancien CEO de Stability AI, va encore plus loin en prédisant un monde sans développeurs d'ici cinq ans

- Jensen Huang, CEO de Nvidia, a déclaré que "tout le monde est désormais développeur" grâce aux assistants IA

Les IDE existants pour Python

- **Cursor**
 - Basé sur VSCode qui intègre de l'IA gen (comme Copilot, codium ou Supermaven mais en mieux)
 - Prise en main rapide et reprise de l'environnement de VSCode existant
 - Modèle freemium
 - Racheté par SpaceX en juin 2026 pour 60 milliards \$



Comment choisir son IDE ?



Le choix d'un IDE dépend souvent :

- **Préférences personnelles et expérience du développeur**
- **Préférences de l'entreprise**
- **Taille du projet**
- **Coût**
- **Caractéristiques et outils**
 - **Débogage** : La capacité à définir des points d'arrêt, à inspecter les variables et à suivre le flux d'exécution
 - **Intelligence du code** : complétion automatique, suggestions de code, mise en évidence des erreurs, etc.
 - **Intégration Git** : une intégration directe dans l'IDE peut être utile.
 - **Gestion des environnements virtuels**: La prise en charge de `venv` ou `conda`.
- **Plateforme et interopérabilité**
 - Pensez à l'OS que vous utilisez (Windows, macOS, Linux)
 - Il existe des IDE qui prennent en charge plusieurs langages.
- **Performance**
 - Certains IDEs peuvent être lourds et prendre beaucoup de ressources.
- **Communauté et support**
 - plus de plugins/extensions, une meilleure documentation et un support plus réactif.

Visual Studio

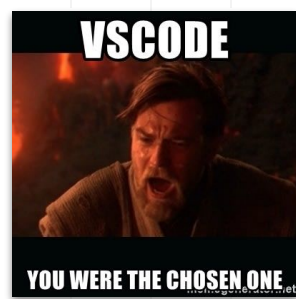
Visual Studio Code, souvent abrégé en VSCode, est un éditeur de code source relativement **léger** mais **puissant**

Avantages

- **Léger et rapide**
- **Extensions** : il existe des milliers d'extensions disponibles pour VSCode (via le Visual Studio Code Marketplace)
 - extension jupyter pour ouvrir / exécuter des notebooks comme dans un navigateur
 - possibilité d'exécuter des cellules interactives dans un fichier .py quelconque en ajoutant `#%%` dans le fichier
 - etc.
- **Intégration Git** : facilite le suivi des modifications, les commits, les pulls, etc.
- **Débogage** : fixer des points d'arrêt, inspecter des variables, contrôler l'exécution du code, etc.
- **Thèmes et personnalisations** : pour adapter l'éditeur à vos préférences.
- **Terminal intégré** : vous pouvez ouvrir un terminal directement dans VSCode, ce qui facilite l'exécution de scripts, l'installation de paquets ou la gestion de votre environnement de développement.
- **Support Multi-langage**
- **Gratuit et Open Source**



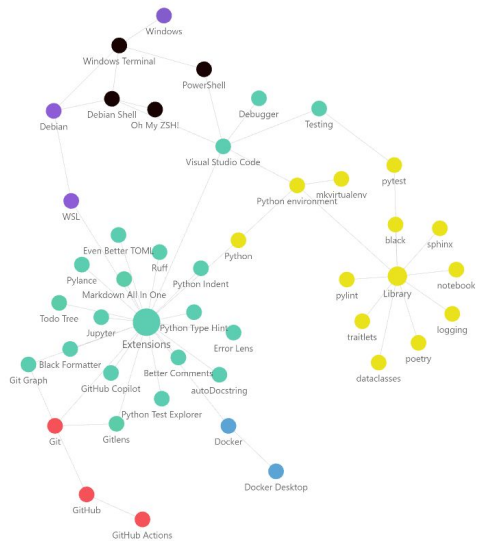
Visual Studio



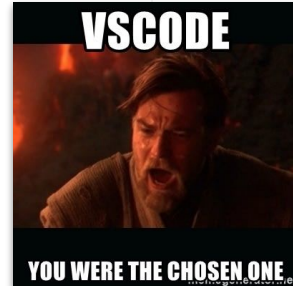
Inconvénients

- **Moins de fonctionnalités spécialisées** par rapport à des IDE dédiés comme PyCharm
- **Courbe d'apprentissage** qui nécessite un certain temps d'adaptation
- **Dépendance aux Extensions** : pour tirer le meilleur parti de VSCode pour le développement Python, vous devez souvent installer et configurer plusieurs extensions.
- **Consommation de mémoire** : l'ajout de multiples extensions peut augmenter sa consommation de mémoire.

Python environment

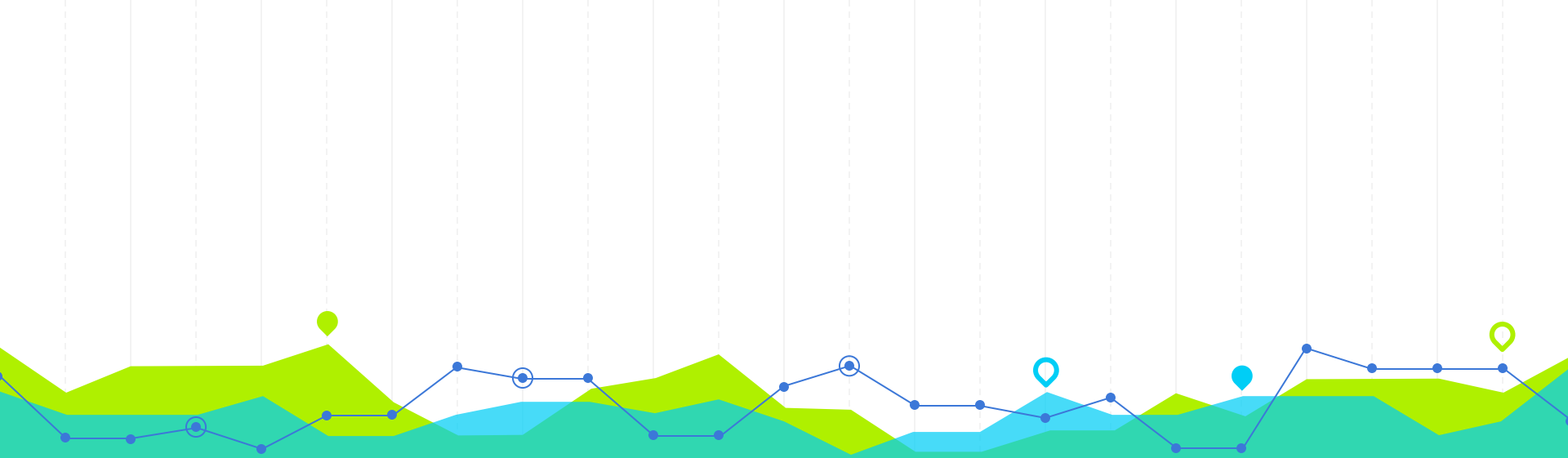


Extensions Visual Studio



- **Even Better TOML** : édition améliorée des fichiers TOML.
- **Ruff** : linter rapide pour Python.
- **Pylance** : typage et intelligence avancés pour Python.
- **Python Indent** : indentation automatique pour Python.
- **Markdown All in One** : outils complets pour Markdown.
- **Todo Tree** : affiche les TODOs et annotations.
- **Jupyter** : support des notebooks Jupyter.
- **Python Type Hint** : indications visuelles pour les types Python.
- **Error Lens** : mise en surbrillance des erreurs et avertissements.
- **Black Formatter** : formatage automatique avec Black.
- **Git Graph** : visualisation des branches et commits Git.
- **GitLens** : informations approfondies sur Git.
- **GitHub Copilot** : suggestions de code basées sur l'IA.
- **Better Comments** : amélioration des commentaires avec des couleurs.
- **AutoDocstring** : génération automatique de docstrings.
- **Python Test Explorer** : gestion des tests Python.
- **Peacock** : colorisation de vos fenêtres en fonction des projets





2. Configurer son IDE

Installer Visual Studio Code

Installation de Visual Studio Code:

- Téléchargez et installez VS Code depuis le site officiel : <https://code.visualstudio.com/>

Installer l'extension Python:

- Ouvrez VS Code.
- Sur la barre latérale, cliquez sur l'icône des extensions
- Cherchez "Python" dans la barre de recherche et installez l'extension officielle publiée par Microsoft.



**Plain
VSCode**



**VSCode with
extensions**



Configurer Visual Studio

- **Configurer l'interpréteur Python**

- ouvrez un fichier `.py`.
- Dans la barre inférieure droite, cliquez sur "Select Python Interpreter".
- Sélectionnez l'interpréteur que vous souhaitez utiliser pour votre projet. Si vous n'avez pas encore installé Python, faites-le à partir de <https://www.python.org/downloads/>.
vous devrez peut-être ajouter son chemin d'accès à votre variable d'environnement PATH ou utiliser le paramètre `python.pythonPath` dans les préférences de VS Code.

- **Formattage du code (PEP 8)**

- installer un outil de formatage automatique comme `black`, `autopep8` et `yapf` (par exemple : `pip install autopep8`)
- allez dans les paramètres (fichier -> préférences -> paramètres) et recherchez "Python Formatting Provider". Sélectionnez le formateur que vous avez installé.
- Pour formater automatiquement le code à chaque enregistrement, réglez "Editor: Format On Save" sur `true`.

- **Linting**

- Le linting vérifie votre code pour détecter les erreurs potentielles et les améliorations suggérées.
- Par défaut, VS Code utilise `pylint`. Vous pouvez l'installer avec `pip install pylint`.
- Si vous préférez un autre linter, configurez-le dans les paramètres de VS Code.

Visual Studio

- **Gestion des environnements virtuels**

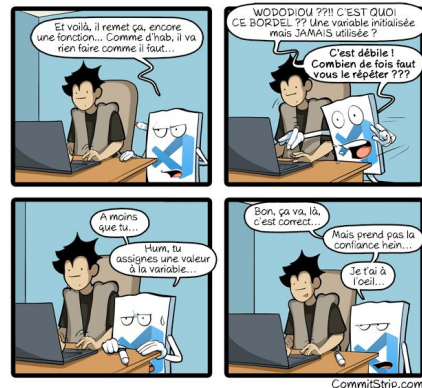
- Il est recommandé d'utiliser des environnements virtuels pour chaque projet afin d'isoler les dépendances.
- Vous pouvez créer un environnement virtuel avec **venv**, **uv** ou **conda**.
- Une fois créé, sélectionnez l'interpréteur de cet environnement comme décrit à l'étape interpréteur Python.

- **Extensions supplémentaires**

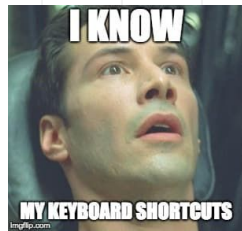
- Selon vos besoins, considérez d'autres extensions telles que **Python Docstring Generator**, **Python Test Explorer**, et bien d'autres pour améliorer votre flux de travail.

- **Apprenez les raccourcis clavier**

- **Ctrl + Space** pour les suggestions d'achèvement de code, ou **F12** pour aller à la définition d'une fonction.



CommiStrip.com



Configuration de Visual Studio

Liste des raccourcis les plus utiles pour coder proprement et efficacement en Python avec VS Code :

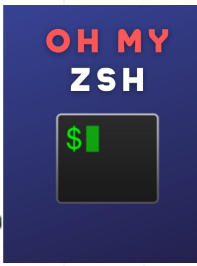
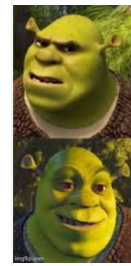
- **Navigation et Edition**
 - **Ctrl + P** : Accédez rapidement à n'importe quel fichier.
 - **Ctrl + F** : Recherche dans le fichier actuel.
 - **Ctrl + H** : Remplacer dans le fichier actuel.
 - **Ctrl + Shift + F** : Recherche globale dans tous les fichiers.
 - **Alt + ↑ / ↓** : Déplace la ligne actuelle vers le haut ou le bas.
 - **Shift + Alt + ↑ / ↓** : Duplique la ligne actuelle vers le haut ou le bas.
 - **Ctrl + D** : Sélectionne la prochaine occurrence du mot sous le curseur.
 - **Ctrl + X** : Coupe la ligne actuelle (ou la sélection).
- **Refactoring et Automatisation**
 - **F2** : Renommer le symbole sous le curseur (comme une variable, une fonction, etc.).
 - **Alt + Entrée** : Affiche les actions de refactoring disponibles.
 - **Ctrl + Espace** : Déclenche l'auto-complétion.
- **Gestion des erreurs et des warnings**
 - **F8** : Naviguer entre les erreurs et les warnings.
 - **Ctrl + .** : Affiche les solutions suggérées pour les problèmes détectés par l'extension Python.



Configuration de Visual Studio

- **Débogage**
 - **F5** : Démarrer/Continuer le débogage.
 - **F10** : Exécuter la ligne actuelle et passer à la suivante en mode débogage.
 - **F11** : Entrer dans une fonction en mode débogage.
 - **Shift + F11** : Sortir d'une fonction en mode débogage.
 - **Ctrl + Shift + F5** : Redémarrer la session de débogage.
- **Gestion des terminaux et des environnements virtuels**
 - **Ctrl + `** : Afficher/Masquer le terminal.
 - **Ctrl + Shift + (backtick)** : Ouvrir un nouveau terminal.
 - **Ctrl + Shift + P** puis tapez "Python: Select Interpreter" : Pour sélectionner un interpréteur Python spécifique ou un environnement virtuel.
- **Gestion des extensions**
 - **Ctrl + Shift + X** : Ouvrir le panneau des extensions.
- **Gestion de l'interface**
 - **Ctrl + B** : Afficher/Masquer la barre latérale.
 - **Ctrl + W** : Fermer le fichier ou l'onglet actuel.
 - **Ctrl + K Z** : Mode zen (plein écran sans distraction).

Terminal



Oh My Zsh est un gestionnaire de configuration pour le shell Zsh, qui est une **alternative au shell Bash**.

Il simplifie la personnalisation de l'interface en fournissant un ensemble de **thèmes**, de **plugins** et de **raccourcis utiles**.

Avantages :

- améliorer l'efficacité de votre travail en ligne de commande
- bénéficier d'une meilleure expérience utilisateur
- gagner du temps via la saisie semi-automatique, la gestion git, ou la complétion intelligente

```
ohmyzsh demo

-> projects take omz-demo && git init
Initialized empty Git repository in /Users/robbyrussell/projects/omz-demo/.git/
-> omz-demo git:(main) echo "TODO: this is my new README" > README.md
-> omz-demo git:(main) git add README.md
-> omz-demo git:(main) # git commit -m "Adding a README to new repo" --quiet
-> omz-demo git:(main) echo "Wow, Oh My Zsh looks neat" >> README.md
-> omz-demo git:(main) # git add -p
diff --git a/README.md b/README.md
index fc97e00..b0939f1 100644
--- a/README.md
+++ b/README.md
@@ -1,1,2 @@
TODO: this is my new README
Wow, Oh My Zsh looks neat
(1/1) Stage this hunk [y,n,q,a,d,e,f]? y

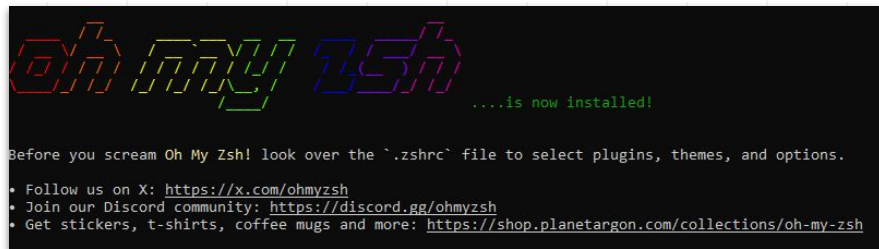
-> omz-demo git:(main) # git commit -m "Updating README in the repo" --quiet
-> omz-demo git:(main) echo ".env" >> .gitignore
-> omz-demo git:(main) git add .gitignore
-> omz-demo git:(main) # git commit -m "Ignoring .env file" --quiet
-> omz-demo git:(main) git checkout -b feature/openai-integration
Switched to a new branch 'feature/openai-integration'
-> omz-demo git:(feature/openai-integration) echo "OPENAI_API_KEY=nd13u9fg23f9hd2p3" >> .env
-> omz-demo git:(feature/openai-integration) git checkout main
Switched to branch 'main'
-> omz-demo git:(main) ..
-> projects []
```

```
-> mkdir zshhh
-> cd zshhh
~/zshhh git init
Initialized empty Git repository in /home/mitchel/zshhh/.git/
~/zshhh master touch zshhh.txt
~/zshhh master gaa
~/zshhh master gcam "first commit"
(master root-commit) 40ef851 first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 zshhh.txt
~/zshhh master Zsh is great!
zsh: command not found: Zsh
~/zshhh master
```

Terminal

Pour l'installer :

```
sudo apt update  
sudo apt install zsh  
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"  
chsh -s $(which zsh) # pour définir zsh par défaut
```



WSL (pour Windows)

WSL permet aux développeurs d'installer une distribution Linux (comme Ubuntu, OpenSUSE, Kali, Debian, Arch Linux, etc.) et d'utiliser des applications, des utilitaires et des outils en ligne de commande Bash **Linux directement sur Windows**

1. Installez Windows Subsystem for Linux (WSL) : <https://learn.microsoft.com/en-us/windows/wsl/install>
`wsl --install`
2. Optionnel : sélectionnez une distribution Linux : ouvrez Microsoft Store et recherchez une distribution Linux de votre choix (comme Ubuntu, Debian, ou autre) et installez-la.
3. Lancez WSL depuis le menu Démarrer ou en tapant son nom dans la recherche

```
C:\Users\prill>wsl --list --online
Voici une liste des distributions valides qui peuvent être installées.
Installer en utilisant 'wsl.exe --install <Distro>'.

NAME                                FRIENDLY NAME
Ubuntu                              Ubuntu
Debian                              Debian GNU/Linux
kali-linux                          Kali Linux Rolling
Ubuntu-18.04                        Ubuntu 18.04 LTS
Ubuntu-20.04                        Ubuntu 20.04 LTS
Ubuntu-22.04                        Ubuntu 22.04 LTS
Ubuntu-24.04                        Ubuntu 24.04 LTS
OracleLinux_7.9                    Oracle Linux 7.9
OracleLinux_8.7                    Oracle Linux 8.7
OracleLinux_9.1                    Oracle Linux 9.1
openSUSE-Leap-15.6                  openSUSE Leap 15.6
SUSE-Linux-Enterprise-15-SP5        SUSE Linux Enterprise 15 SP5
SUSE-Linux-Enterprise-Server-15-SP6 SUSE Linux Enterprise Server 15 SP6
openSUSE-Tumbleweed                 openSUSE Tumbleweed

C:\Users\prill>wsl --install debian
Installation : Debian GNU/Linux
Debian GNU/Linux a été installé.
Lancement de Debian GNU/Linux...
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: martin
New password:
Retype new password:
```





À votre tour

Installer Visual Studio Code

Installation de Visual Studio Code:

- Téléchargez et installez VS Code depuis le site officiel : <https://code.visualstudio.com/>.

Installer l'extension Python:

- Ouvrez VS Code.
- Sur la barre latérale, cliquez sur l'icône des extensions (qui ressemble à des blocs).
- Cherchez "Python" dans la barre de recherche et installez l'extension officielle publiée par Microsoft.

Configurer l'interpréteur Python:

- Une fois l'extension Python installée, ouvrez un fichier `.py`.
- Dans la barre inférieure droite, sélectionnez l'interpréteur que vous souhaitez utiliser pour votre projet. Si vous n'avez pas encore installé Python, faites-le à partir de <https://www.python.org/downloads/>. vous devrez peut-être ajouter son chemin d'accès à votre variable d'environnement PATH ou utiliser le paramètre `python.pythonPath` dans les préférences de VS Code.

Installer Visual Studio Code

Formattage du code (PEP 8):

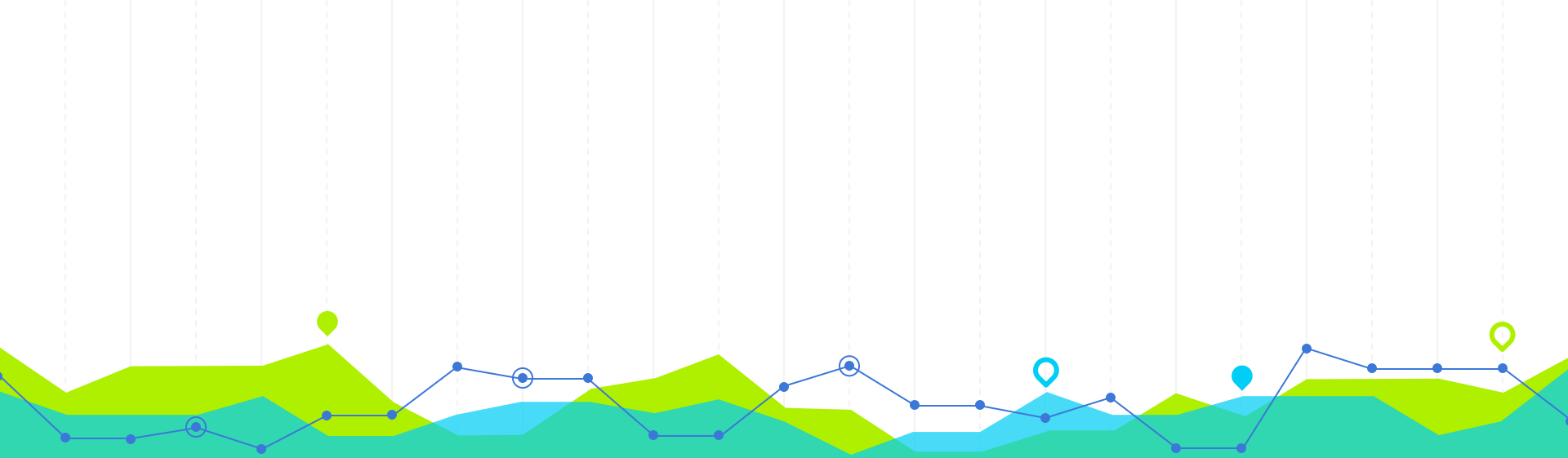
- Pour maintenir un code propre et conforme à la convention PEP 8 de Python, utilisez un outil de formattage automatique comme `black`, `autopep8` et `yapf`.
- Installez votre formateur préféré avec pip, par exemple: `pip install autopep8`.
- Dans VS Code, allez dans les paramètres (fichier -> préférences -> paramètres) et recherchez "Provider". Sélectionnez le formateur que vous avez installé (`autopep8` dans cet exemple).
- Pour formater automatiquement le code à chaque enregistrement, réglez "Editor: Format On Save" sur true.

Linting

- Le linting **vérifie** votre code pour détecter les erreurs potentielles et les améliorations suggérées.
- Par défaut, VS Code utilise `pylint`. Vous pouvez l'installer avec `pip install pylint`.

Extensions supplémentaires

- `Python Docstring Generator`, `Python Test Explorer`, et bien d'autres pour améliorer votre flux de travail.



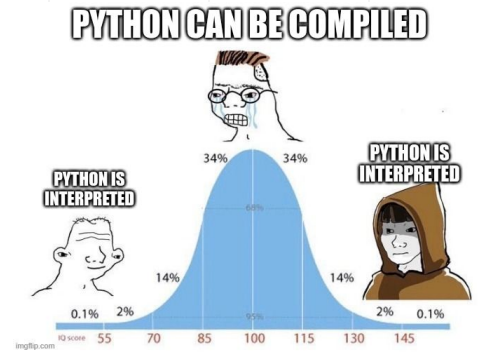
3. Choisir sa version de Python

Comment marche Python ?

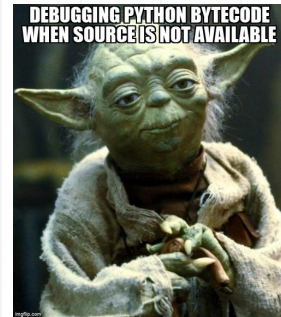
Python est un langage de programmation de haut niveau qui est **interprété**

Voici comment cela fonctionne étape par étape :

1. **Écriture du code source** : en texte brut, généralement en utilisant l'extension `.py` pour le fichier.
2. **Compilation en bytecode** : Quand vous exécutez un fichier Python avec l'**interpréteur Python**, le code est d'abord compilé en "**bytecode**" : une représentation de bas niveau, **plateforme-indépendante**, de votre code source. Ces fichiers bytecode ont généralement l'extension `.pyc` et sont stockés dans le dossier `__pycache__`.
3. **Interprétation du bytecode** : la machine virtuelle Python (PVM) lit et exécute ensuite ce bytecode. C'est cette étape qui est souvent qualifiée d'"interprétation", car la PVM interprète le bytecode et effectue les opérations appropriées pour votre système d'exploitation et votre architecture matérielle.



Comment marche Python ?



Ce processus peut sembler comporter un surcoût par rapport à la compilation directe en code machine (comme c'est le cas avec des langages comme C ou C++), mais il présente des avantages :

- **Portabilité** : le bytecode est **indépendant de la plateforme**.
Vous pouvez donc écrire du code Python sur une machine, le compiler en bytecode, et exécuter ce bytecode sur une autre machine avec une PVM compatible
- **Facilité de développement** : l'interprétation permet des cycles de développement rapides, car pas besoin d'un processus de compilation complet pour tester les changements
- **Flexibilité** : l'approche interprétée permet des fonctionnalités dynamiques difficiles à réaliser avec des langages compilés
Ex : comme l'évaluation dynamique du code

Il existe aussi des outils qui peuvent "**compiler**" le **code Python en code machine** pour des raisons de performance ou de distribution, comme **Cython**, **PyInstaller**, et **Nuitka**

Comment marche Python ?

Python

- est livré avec une vaste **bibliothèque standard**
- utilise un **compteur de référence** pour chaque objet en **mémoire**. Lorsqu'un objet n'est plus référencé, sa mémoire est libérée
- détermine la portée d'un bloc de code via **l'indentation**
- est **typé dynamiquement** -> le type d'une variable est déterminé à l'exécution
- utilise l'**interpréteur Python CPython** de référence. Il existe un **verrou global** appelé **GIL (Global Interpreter Lock)** qui empêche plusieurs threads natifs d'exécuter du bytecode Python en même temps. Cela peut limiter l'utilisation multi-cœur pour les applications CPU-bound
- peut être **étendu avec du code en C** pour augmenter ses performances ou pour accéder à des bibliothèques écrites dans d'autres langages

Python est un langage polyvalent, adapté à une grande variété de tâches, des scripts simples aux applications Web complexes





Qu'est ce qu'une version de Python ?

Une version de Python désigne une sortie spécifique du langage de programmation Python :

- **Python 1.x** : première version de Python (Python 0.9.0), publiée en 1991 par Guido van Rossum. Python 1.0 a officiellement vu le jour en janvier 1994
- **Python 2.x** : publié en 2000. Cette version a introduit de nombreuses fonctionnalités du langage qui étaient auparavant absentes, notamment les list comprehensions et le garbage collection. **Python 2.7**, publié en 2010, était la dernière version de la série 2.x. Cette version était largement adoptée dans l'industrie et a bénéficié d'un support prolongé **jusqu'en 2020**.
- **Python 3.x** : Publié pour la première fois en **2008**, Python 3 était une refonte majeure du langage, visant à corriger plusieurs défauts structurels, même au prix de la compatibilité ascendante. Python 3 a introduit des fonctionnalités comme la fonction `print()` (au lieu de la déclaration `print`), une nouvelle syntaxe pour les exceptions, et la distinction entre les types `bytes` et `str`. La transition de Python 2 à 3 a été **lente**, mais Python 3 a été **largement** adopté avec le temps.

Qu'est ce qu'une version de Python ?

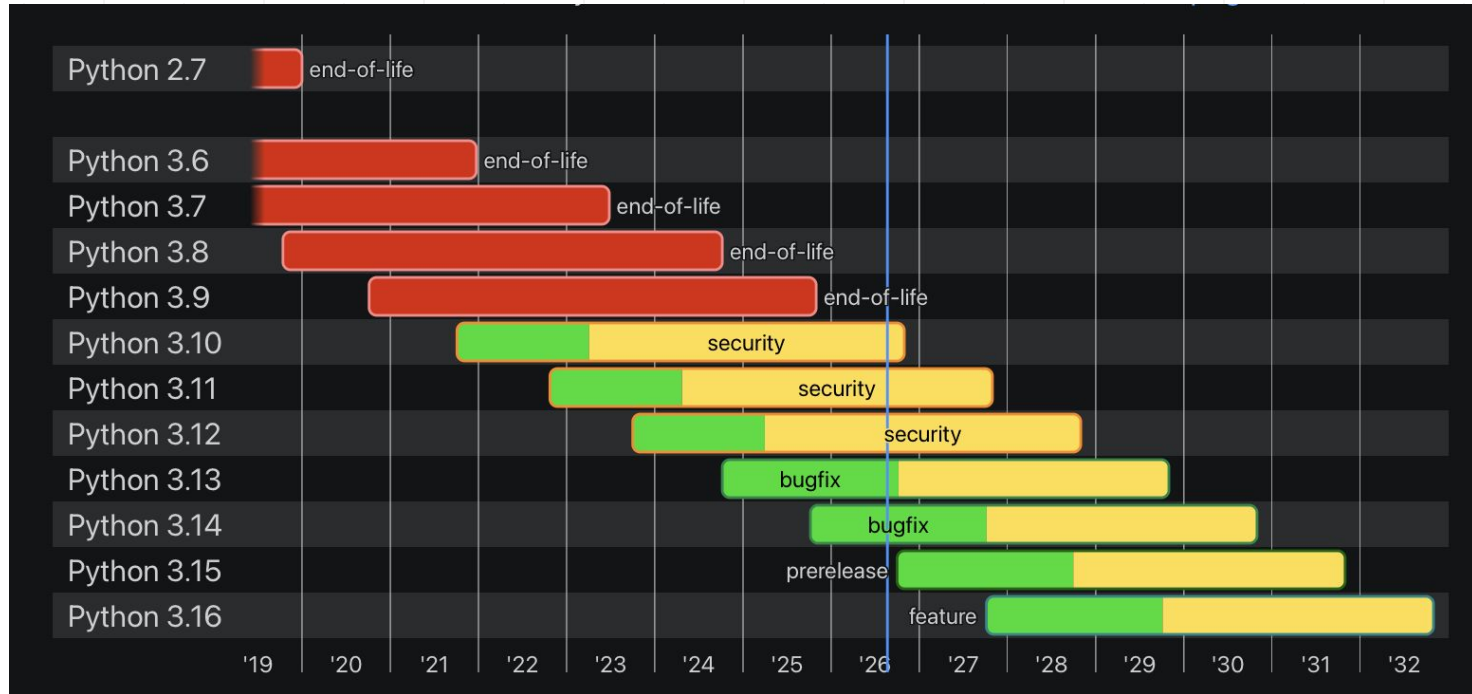


D'autres éléments à considérer

- **Support à long terme (LTS)** : Certaines versions de Python sont désignées comme des versions à "support à long terme". Ces versions reçoivent des correctifs et des mises à jour de sécurité pendant une période prolongée par rapport aux versions normales.
- **PEPs** : Python utilise un processus d'amélioration (**Python Enhancement Proposals** ou PEPs) pour proposer et documenter les changements majeurs au langage. Par exemple, le passage de Python 2 à Python 3 a été détaillé dans le **PEP 3000**.
- **Cadence de sortie** : Historiquement, Python avait une cadence de sortie irrégulière. Cependant, à partir de Python 3.9, il a été décidé que Python aurait des **sorties annuelles**, rendant les sorties plus prévisibles.

Notez que des versions mineures et des correctifs sont **publiés régulièrement** pour corriger les bugs et les problèmes de sécurité.

Qu'est ce qu'une version de Python ?



Comment choisir sa version ?



Considérations pour vous aider à choisir la bonne version de Python pour votre projet :

- **Projet existant vs. Nouveau projet:**
 - sur un **projet existant**, il est conseillé de continuer avec la version de Python déjà utilisée, sauf s'il y a une nécessité impérieuse de mise à jour ou si vous êtes en python 2
 - sur un **nouveau projet**, vous devriez envisager d'utiliser une des **dernières versions stables** de Python (par exemple pas en dessous de la 3.12)
- **Compatibilité avec les bibliothèques:**
 - Avant de choisir une version spécifique de Python 3, vérifiez la **compatibilité** des bibliothèques et des frameworks que vous prévoyez d'utiliser. Certains d'entre eux peuvent **ne pas être compatibles** avec les dernières versions de Python.

Comment choisir sa version ?



Considérations de sécurité

- Utilisez toujours une version de Python qui reçoit encore des mises à jour de sécurité

Performances

- Si la vitesse est critique pour votre application, cela pourrait être une raison d'opter pour une version plus récente

Fonctionnalités spécifiques

- Les nouvelles versions de Python introduisent de nouvelles fonctionnalités et améliorations

Environnement d'exécution

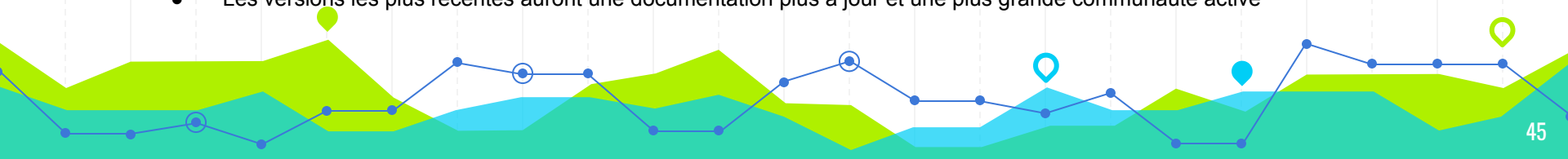
- Si vous déployez votre application dans un environnement spécifique, comme une certaine distribution Linux, AWS Lambda, ou un autre environnement cloud, vous devrez peut-être vous conformer à certaines versions

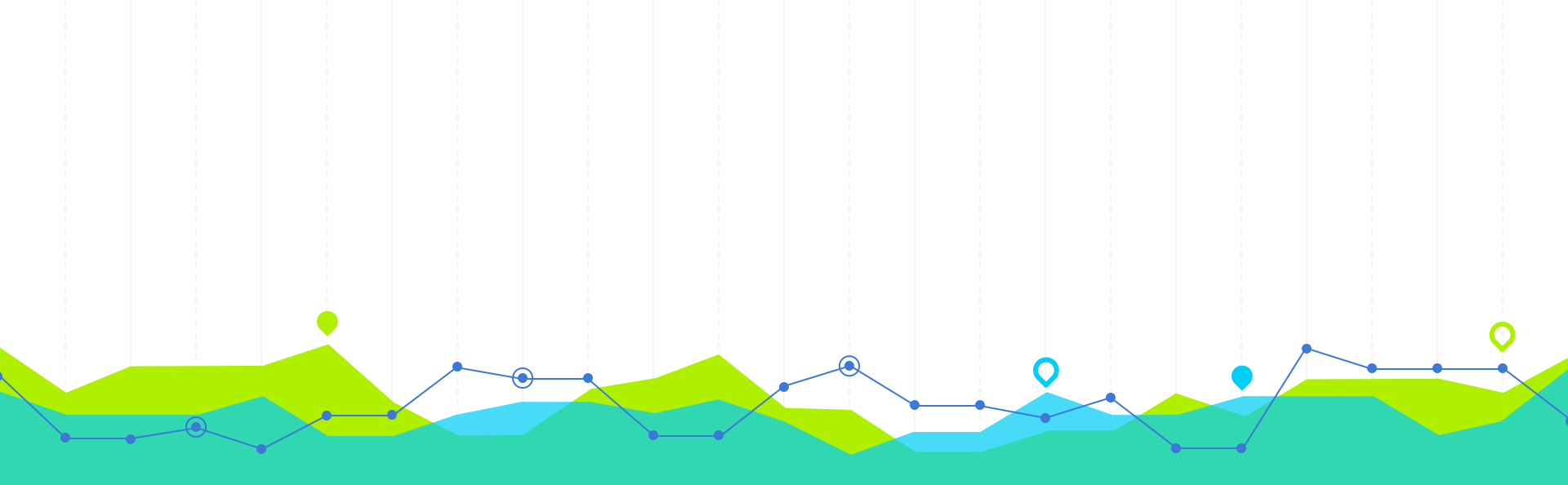
Outils et IDE

- Assurez-vous que vos outils de développement (IDE, linters, débogueurs) sont compatibles

Documentation et communauté

- Les versions les plus récentes auront une documentation plus à jour et une plus grande communauté active





4. Créer ses environnements Python

Qu'est ce qu'un environnement Python ?



=> espace contenant une copie spécifique de Python ainsi que des ensembles de bibliothèques et dépendances, séparés les uns des autres pour éviter tout conflit ou incompatibilité.

Pourquoi utiliser des environnements Python

- **Isolation** : vous pouvez avoir différentes versions d'une bibliothèque pour différents projets
- **Propreté** : vous pouvez supprimer un environnement entier quand vous avez terminé avec un projet, sans affecter d'autres projets
- **Gestion des dépendances** : plus facile de savoir quelles bibliothèques et quelles versions sont nécessaires pour un projet spécifique

Les types d'environnements Python



Types d'environnements Python

Natif (librairies standard) :

- **venv** : module de création d'environnements virtuels intégré à Python. Il est simple et fait partie de la bibliothèque standard de Python.
- **(pyvenv** : natif mais supprimé depuis 3.8)

À installer (librairies PyPI) :

- **virtualenv** : application tierce qui fournit des fonctionnalités supplémentaires par rapport à venv.
- **virtualenvwrapper** : extension de virtualenv, qui fournit des commandes pour créer, supprimer et copier des env virtuels.
- **pyenv** : utilisé pour isoler et tester son code sur différentes versions de Python (il existe aussi pyenv-virtualenv et pyenv-virtualenvwrapper)
- **conda** : système de gestion de paquets et d'environnement. Il peut gérer les dépendances non Python.
- **pipenv** : combine pipfile, pip et virtualenv en une commande. Utilisé lors du développement d'applications Python (par opposition aux bibliothèques)
- **poetry** : gestion des dépendances et des environnements avec un fichier [pyproject.toml](https://pyproject.org/).
- **hatch** : gestion d'env (plusieurs possible par projet), build (wheels / sdist), PyPi, gen auto de squelette de projet, etc.
 - combo gagnant en 2025 : hatch + uv

venv



venv, recommandé pour la plupart des utilisations, est inclus dans la bibliothèque standard à partir de Python 3.3.

1. Ouvrez votre terminal ou invite de commande.
Accédez au dossier où vous souhaitez créer votre environnement virtuel.
Exécutez la commande suivante :

```
python3 -m venv [nom_de_l_environnement]
```

2. Activation et désactivation de l'environnement:

Sur Windows:

```
[nom_de_l_environnement]\Scripts\activate  
# Pour désactiver  
deactivate
```

Sur macOS/Linux:

```
source [nom_de_l_environnement]/bin/activate  
# Pour désactiver  
deactivate
```

virtualenv



virtualenv est une application tierce qui fournit des fonctionnalités supplémentaires par rapport à venv.

1. Installez virtualenv:

```
pip install virtualenv
```


Accédez au dossier où vous souhaitez créer votre environnement virtuel.
2. Créez l'environnement:

```
virtualenv [nom_de_l_environnement]
```
3. Activation et désactivation de l'environnement sont les mêmes que pour venv.

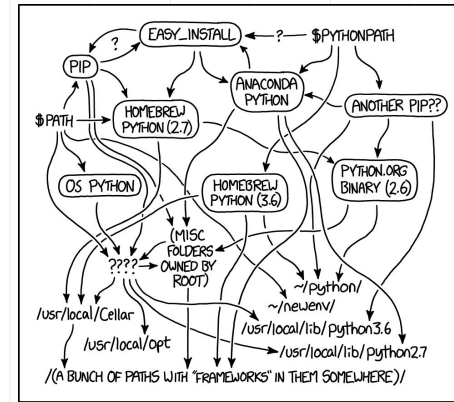
virtualenvwrapper

mkvirtualenv est un ensemble d'extensions de virtualenv, qui fournit des commandes pour créer, supprimer et copier des environnements virtuels. Il fournit également des commandes pour naviguer entre différents environnements virtuels.

1. Installez **virtualenvwrapper**:
Pour les utilisateurs de macOS/Linux:
`pip install virtualenvwrapper`

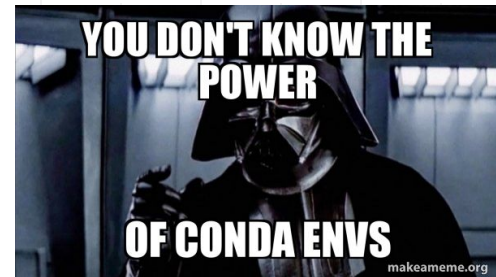
Pour les utilisateurs de Windows:
`pip install virtualenvwrapper-win`
2. Une fois installé, vous pouvez utiliser la commande **mkvirtualenv** pour créer un environnement:
`mkvirtualenv [nom_de_l_environnement]`
3. Utilisez **workon** pour activer un environnement:
`workon [nom_de_l_environnement]`

Pour désactiver l'environnement, vous pouvez simplement utiliser:
`deactivate`



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

Conda



conda est un gestionnaire de paquets et d'environnements, particulièrement populaire dans le monde de la science des données car il peut gérer des paquets non seulement de Python, mais aussi des dépendances non-Python.

1. Installez Anaconda ou Miniconda

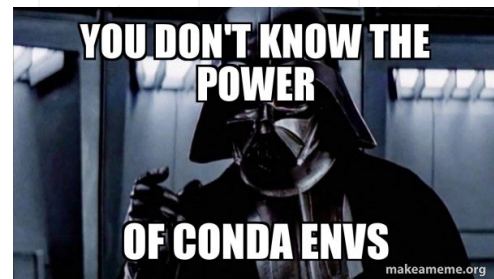
2. Créez l'environnement:

```
conda create --name [nom_de_l_environnement] python=3.8
```

3. Activation et désactivation de l'environnement:

```
conda activate [nom_de_l_environnement]
```

```
conda deactivate # Pour désactiver
```



uv est un **gestionnaire de projets et de dépendances Python** ultra-rapide, écrit en Rust. Il permet de remplacer plusieurs outils comme pip, venv et une partie de poetry avec une seule commande.

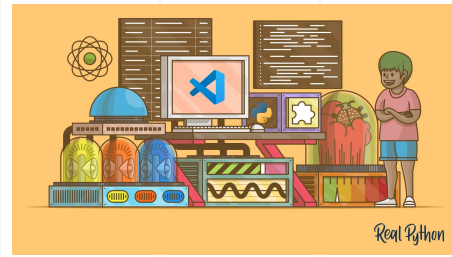
- Linux/ macOS

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- Vérifier l'installation:

```
uv --version
```

Depuis Visual Studio Code (VS Code)



1. Ouvrez votre projet

Ouvrez le dossier de votre projet dans VS Code

2. Sélectionnez un interpréteur Python

- Appuyez sur **Ctrl + Shift + P** pour ouvrir la palette de commandes.
- Tapez "Python: Select Interpreter"
- Sélectionnez l'interpréteur que vous souhaitez utiliser. Si vous avez déjà créé un environnement virtuel pour votre projet, il devrait apparaître dans la liste. Sinon, vous pouvez créer un nouvel environnement virtuel.

3. Utilisation de l'environnement

tout code Python que vous exécutez depuis VS Code utilisera cet environnement

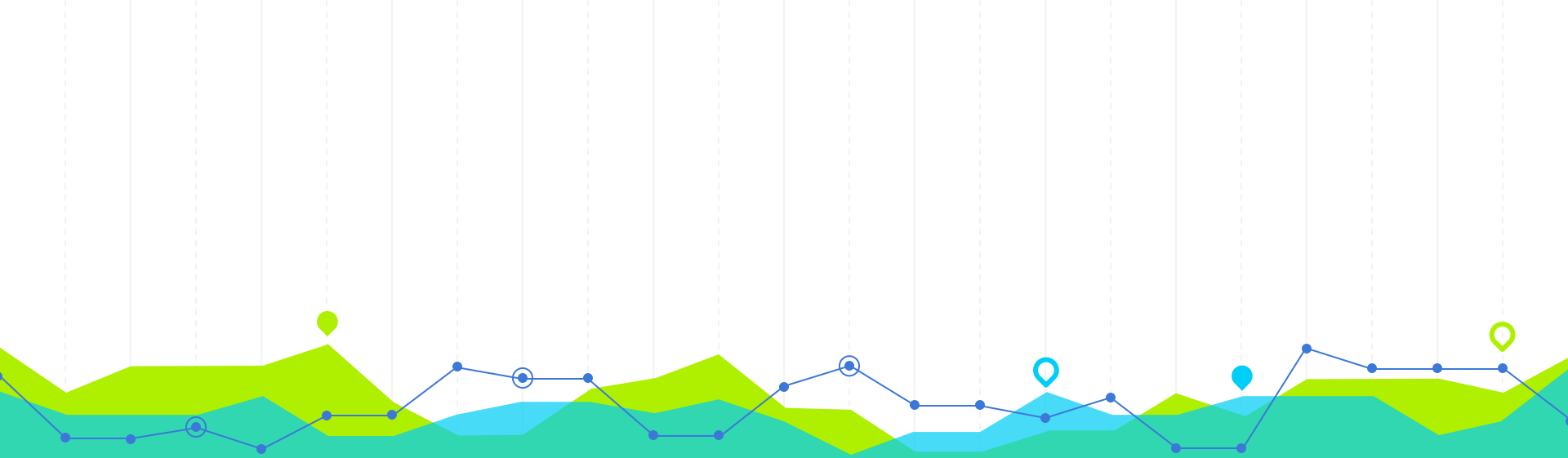
4. Installations de paquets

en utilisant le terminal intégré de VS Code (avec **pip** par exemple), vos paquets seront installés dans l'environnement virtuel actuellement actif.

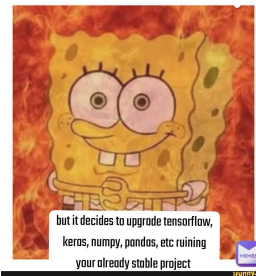
Bonnes pratiques



- Créez un **nouvel environnement** pour chaque **projet** pour garantir l'**isolation** de vos dépendances
- Utilisez des **noms d'environnement descriptifs** de chaque environnement
- **Activez l'environnement** avant de travailler sur un projet
- **Enregistrez vos dépendances** (pyproject.toml)
- Gardez votre **environnement à jour** en mettant à jour les paquets pour bénéficier des derniers fix et améliorations
- Avec des **outils de versionnement (Git)**, ignorez le dossier de l'environnement (.gitignore), mais pas le requirements.txt
- **Documenter** comment configurer l'environnement pour d'autres développeurs ou pour vous-même à l'avenir



5. Choisir ses librairies Python



Qu'est ce qu'une librairie Python ?

"library" / bibliothèque = ensemble de modules (code) que vous pouvez utiliser pour réaliser des tâches spécifiques sans avoir à réécrire le code à chaque fois.

Une librairie est basée sur un thème ou une fonctionnalité spécifique :

- **numpy** est une librairie pour le calcul numérique en Python.
- **requests** est une librairie pour effectuer des requêtes HTTP.
- **pandas (ou mieux: polars)** est une librairie pour la manipulation et l'analyse de données.
- **matplotlib** est une librairie pour la visualisation de données sous forme de graphiques.

Comment installer une librairie ?

1. installation via le système de gestion de paquets **pip**
2. **importer** la librairie dans vos scripts ou notebooks pour utiliser leurs fonctionnalités

Les bibliothèques natives déjà installées avec Python



Create
project
from scratch

Using
python
libraries

"bibliothèque standard" / "standard library" = ensemble de modules et paquets **livrés avec l'installation par défaut de Python.**

Python est donc déjà prêt à l'emploi dans de nombreux scénarios :

- **math** : fournit des fonctions mathématiques
- **datetime** : travailler avec les dates et les temps
- **os** : fonctionnalités d'interaction avec le système d'exploitation, comme la navigation dans le système de fichiers
- **sys** : accéder à certaines variables et fonctions liées à l'interpréteur Python lui-même
- **re** : fournit des fonctions pour travailler avec les expressions régulières
- **json** : encoder et décoder du JSON
- **urllib** : travailler avec les URL, effectuer des requêtes HTTP, etc.
- **sqlite3** : interagir avec les bases de données SQLite
- **random** : des fonctions pour générer des nombres aléatoires
- **collections** : contient des structures de données avancées comme namedtuple, deque, Counter, etc.
- **this**: le "zen" du développement en python, à toujours garder en tête !

... et bien d'autres.

Comment choisir les bonnes librairies ?



Popularité et Communauté

- Utilisateurs actifs = grande communauté = plus d'assistance
- Taux de contributions = régulièrement mise à jour = plus de stabilité / nouveautés

Documentation

- Une bonne documentation facilite l'apprentissage et l'utilisation de la librairie

Compatibilité et Dépendances

- Vérifier que la librairie est compatible avec votre version de Python et que les autres librairies dont elle dépend ne génèrent pas de conflits

Fonctionnalités et Performance

- La librairie offre-t-elle les fonctionnalités dont vous avez besoin ? Est-elle performante pour votre cas ?

Facilité d'utilisation

- Une API intuitive et bien conçue peut grandement réduire le temps de développement

Sécurité

- Assurez-vous que la librairie n'a pas de failles de sécurité connues

Comment choisir les bonnes librairies ?



Licence

- Assurez-vous que la licence de la librairie est compatible avec l'utilisation que vous prévoyez

Tests et stabilité

- Une bonne couverture de tests signifie que la bibliothèque est probablement robuste et moins sujette aux bugs
- Vérifiez également le nombre de bugs ouverts et leur gravité.

Support et Maintenance

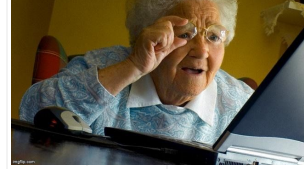
- La librairie est-elle toujours en développement actif ?
- Quelle est la rapidité de réponse aux problèmes et aux questions soulevés ?

Intégration avec d'autres outils

- Dans certains cas, il peut être essentiel que la librairie fonctionne bien avec d'autres outils ou frameworks que vous utilisez

Retours et Avis

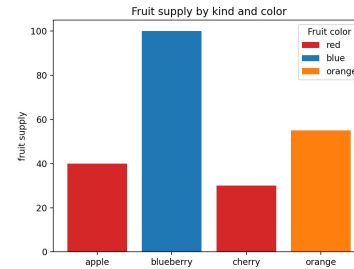
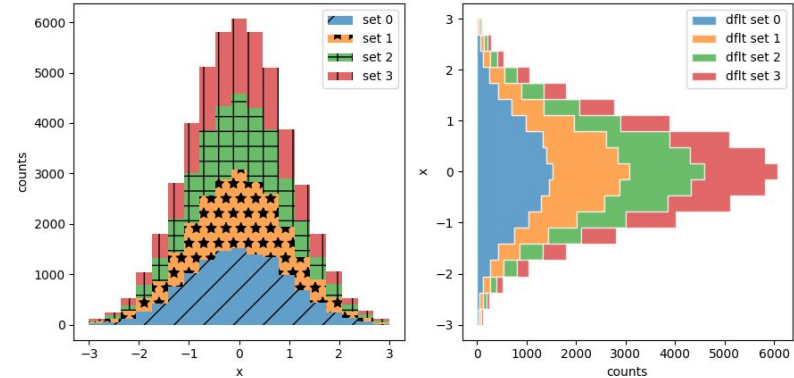
- Les retours d'autres utilisateurs peuvent être précieux. Les forums, les médias sociaux, et les plateformes comme Stack Overflow peuvent vous donner une idée des avantages et des inconvénients de la librairie

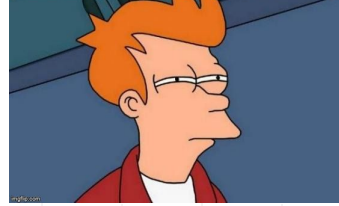


Matplotlib, la librairie historique

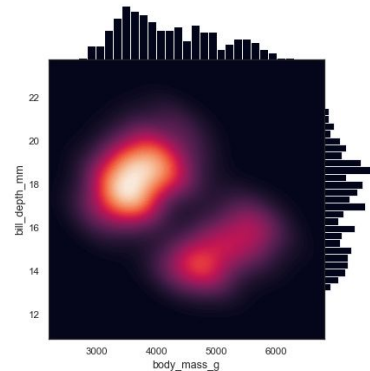
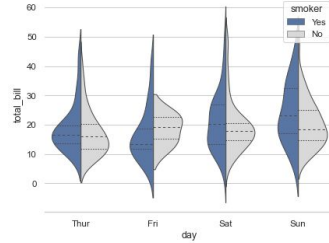
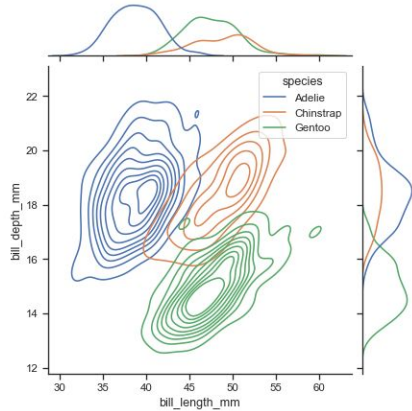
Sobre et sans interactivité mais native dans de nombreuses librairies (comme Pandas ou NetworkX)

https://matplotlib.org/stable/plot_types/index.html





Seaborn, la librairie pour les statistiques



Seaborn est basée sur Matplotlib mais offre une interface plus conviviale et des thèmes harmonisés entre les représentations.

Seaborn est plus adaptée pour des visualisations statistiques.

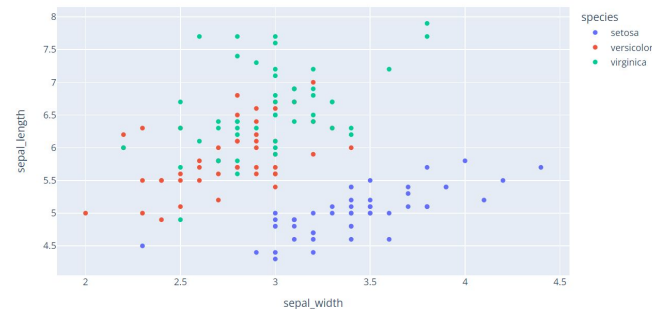
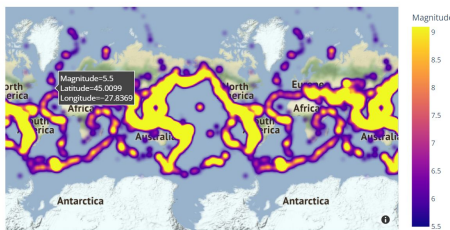
<https://seaborn.pydata.org/examples/index.html>

Plotly, la librairie au goût du jour

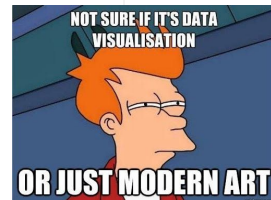


Natif en JavaScript, permet les mêmes visualisation que Matplotlib mais avec l'interactivité en plus et plus esthétique.

<https://plotly.com/python/>

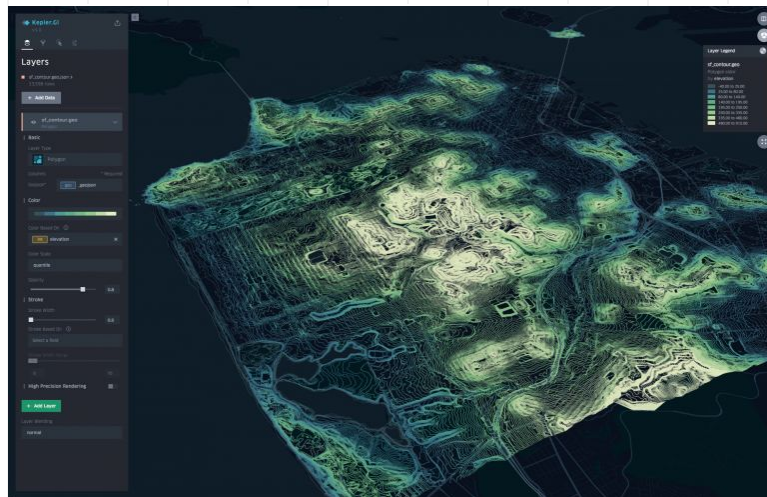


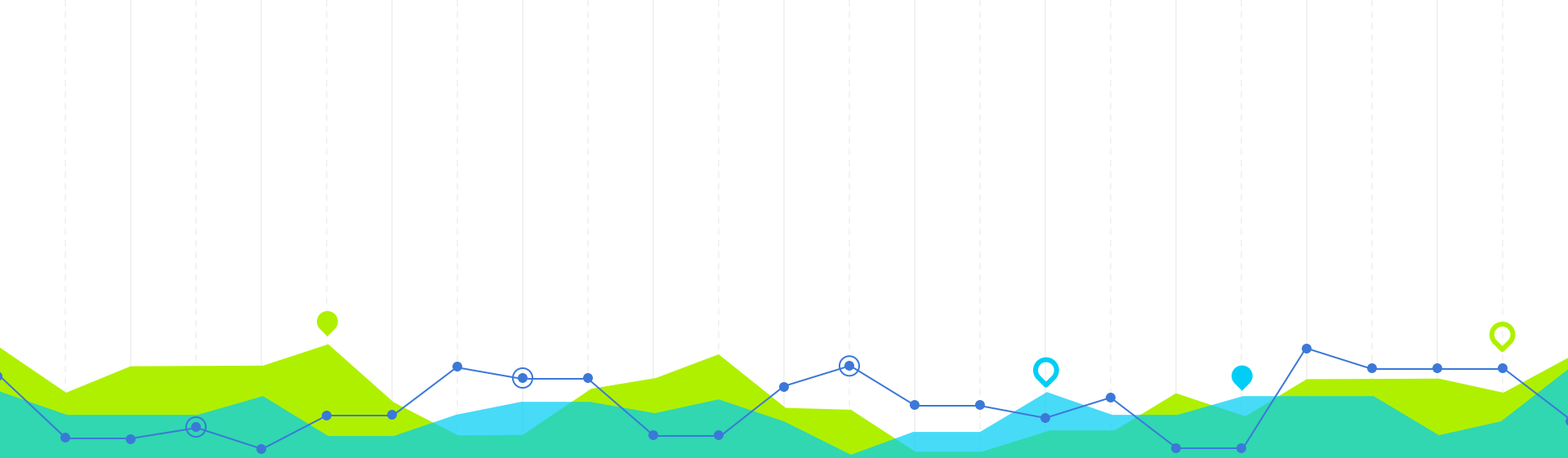
Kepler, la librairie pour visualiser des données géospatiales



Un outil avancé de visualisation géospatiale, pour rendre des cartes interactives.

<https://kepler.gl/>





6. Utiliser le debugger de son IDE

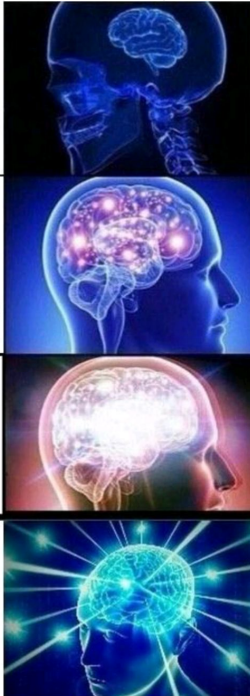
Qu'est ce qu'un debugger ?

UTILISER
LE
DEBUGGEUR PYTHON

UTILISER JUPYTER
POUR EVALUER CHAQUE
CELLULE JUSQU'A CE
QUE CA FONCTIONNE
PUIS COPIER LE CODE EN PROD

```
PRINT("COUCOU")  
PRINT("COUCOU2")  
PRINT("A")  
PRINT("AB")  
PRINT("ABABABA")  
PRINT("ABABABAAAAA")  
PRINT("ABABABSABBSBASBSA")  
PRINT("MERDE")
```

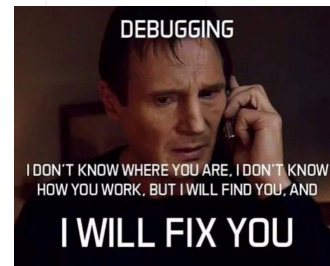
SIMULER LES INPUTS
ET VARIABLES SUR UN PAPIER
AVEC UN CRAYON, GOMMER
A CHAQUE CHANGEMENT DE STATE,
TRANSCRIRE TOUT CA EN
PYTHON, S'APERCEVOIR QU'ON
A OUBLIÉ UN TRUC, RECOMMENCER...



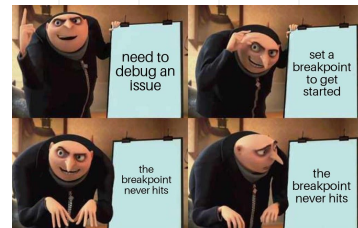
débogage = processus de **détecter**, **localiser** et **corriger** les erreurs ou "bugs" dans un programme

Pourquoi est-ce important ?

- **Gagner du temps** en identifiant précisément **où** et **pourquoi** votre code ne fonctionne pas
- Plonger profondément dans votre code vous donne une **meilleure compréhension** de son fonctionnement interne
- **Confiance accrue** à résoudre des problèmes complexes et des erreurs inattendues
- les bugs sont inévitables, mais un débogage efficace garantit que ces bugs sont **résolus rapidement** et **correctement** = **code plus fiable et robuste**



Qu'est ce qu'un point d'arrêt ?



point d'arrêt = **marqueur** que vous placez sur une ligne de votre code pour indiquer à l'IDE qu'il doit **interrompre** l'exécution à cet endroit.

Il existe des points d'arrêt

- **simples** : points d'arrêt classiques que vous définissez pour **interrompre l'exécution**. Utiles lorsque vous savez exactement où vous voulez vous arrêter
- **conditionnels** : déclenchés seulement si une **condition** spécifique est vraie. Par exemple, vous pouvez avoir une boucle qui s'exécute 1000 fois, mais vous voulez vous arrêter seulement quand la variable `i` est égale à 500
- **de log** : émettent un log ou un message dans la console. Ils sont utiles pour **suivre** le flux d'exécution sans interrompre

Ajouter

Cliquez sur la **marge à côté du numéro de ligne** où vous voulez mettre le point d'arrêt. Un **cercle rouge** apparaîtra.

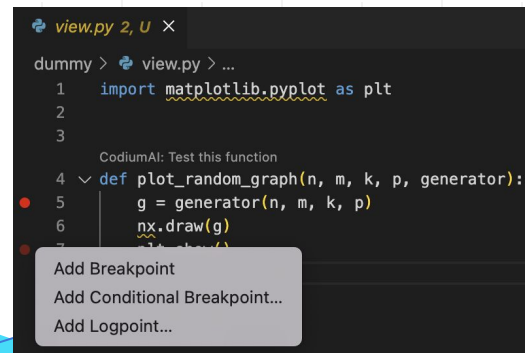
Désactiver ou supprimer

Désactiver temporairement : clique droit sur le point d'arrêt

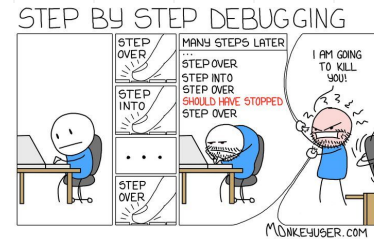
Suppression : cliquez à nouveau sur son icône

Liste des points d'arrêt et navigation

Les IDEs avancés fournissent une fenêtre ou un panneau qui liste tous les points d'arrêt définis.



Exécution pas à pas



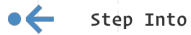
permet au développeur de **contrôler l'exécution** du programme **ligne par ligne** pour comprendre comment le code fonctionne et où il pourrait y avoir des erreurs et pour inspecter l'état du programme (variables, appels de fonction, etc.).



Step Over

Exécute la ligne de code actuelle et se déplace à la **ligne suivante** dans le même fichier.

Quand l'utiliser : Quand vous voulez exécuter la ligne actuelle mais que vous n'êtes pas intéressé à entrer dans une fonction ou une méthode.



Step Into

Si la ligne actuelle contient un appel à une fonction ou à une méthode, "Step Into" vous amènera à **l'intérieur** de cette fonction/méthode.

Quand l'utiliser : Quand vous voulez **comprendre** comment une fonction ou une méthode spécifique fonctionne.



Step Out

Si vous êtes à l'intérieur d'une fonction ou d'une méthode, "Step Out" vous ramènera à **la ligne après l'appel de cette fonction/méthode** dans le fichier d'origine.

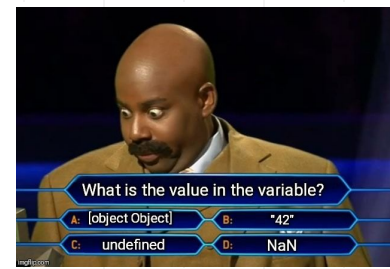
Quand l'utiliser : Quand vous avez **terminé d'inspecter** une fonction et que vous souhaitez retourner à l'endroit où elle a été appelée.



Continue

Exécute le programme normalement **jusqu'au prochain point d'arrêt** ou jusqu'à la fin du programme.

Inspection des variables



L'inspection des variables permet aux développeurs de voir exactement ce qui se passe à l'intérieur de leur programme. C'est la capacité de **visualiser** et **d'interagir** avec les variables et leurs valeurs pendant l'exécution d'un programme.

Visualisation des variables

Panneau des variables ou une fenêtre dédiée à l'affichage des variables (locales, globales, etc.) actuellement en portée. Vous pouvez souvent développer des objets complexes (comme des listes ou des dictionnaires) pour voir leurs éléments.

A screenshot of a Jupyter Notebook interface. On the left, the 'Variables' panel is open, showing 'function variables' with values: k: 5, m: 5, n: 16, p: 0.495. The code editor on the right shows a function definition for 'plot_random_graph' with parameters (n, m, k, p, generator). The function body includes 'g = generator(n, m, k, p)', 'nx.draw(g)', and 'plt.show()'. The interface also shows a 'RUN AND DEBUG' button and a search bar at the top.

Techniques de débogage avancées



print et fonctions de logging

- `print` ou le module `logging` peut fournir de précieuses informations sur l'état d'exécution de votre programme

Profiling

- Si votre code s'exécute lentement, utilisez `cProfile` ou `profile` pour voir où votre code passe le plus de temps

traceback

- Quand une exception est levée, Python print une trace de la pile (stack trace). Consultez-la !

Outils de visualisation:

- Py-spy est un échantillonneur de performance. Il vous permet de visualiser ce que fait votre code en temps réel
- Objgraph peut vous aider à visualiser les références d'objets si vous traquez des fuites de mémoire

Static Code Analysis:

- Des outils comme `pylint` ou `flake8` peuvent aider à identifier les problèmes potentiels dans votre code sans même l'exécuter

Type Checking:

- Avec l'introduction des annotations de type, Python peut bénéficier de la vérification de type à l'aide d'outils comme `mypy`



Techniques de débogage avancées

pdb - Le débogueur intégré de Python

- `pdb` est le débogueur standard de Python.
- Vous pouvez insérer `import pdb; pdb.set_trace()` n'importe où dans votre code pour démarrer une session de débogage
- utiliser des commandes telles que `n` (next), `c` (continue), `s` (step), `l` (list), etc. pour naviguer à travers votre code

Débogage post-mortem avec pdb

- Si votre code génère une exception, utiliser `pdb.post_mortem()` pour lancer une session de débogage au point de l'exception

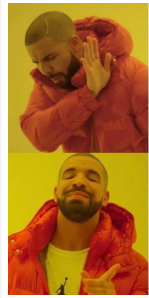
Extensions de débogage

- `ipdb` est une version améliorée de `pdb`, avec notamment une meilleure coloration syntaxique et une auto-complétion.
- Sur Jupyter ou IPython, utiliser la commande `%debug` pour démarrer une session de débogage après une exception

Écriture de tests:

- L'écriture de tests unitaires avec des frameworks tels que `unittest`, `pytest` ou `nose` est un excellent moyen de trouver des bugs. Quand un **test échoue**, vous savez qu'il y a probablement un **bug lié** à la partie du code testée

Astuces et conseils

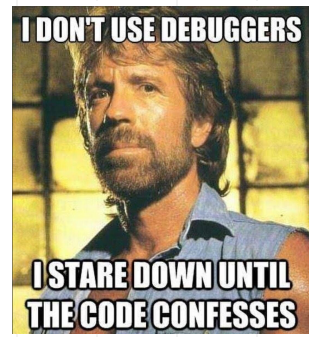


Using a debugger
to find where your
program crashed

print("got this far")

- Avant d'écrire de grandes parties de code, pensez d'abord aux possibles erreurs et pièges (**débogage préventif**)
- **Utilisez des messages de log** au lieu de toujours compter sur les points d'arrêt, incorporez des logs dans votre code
- **Évitez d'utiliser le débogueur pour tout** et concentrez-vous sur certaines parties du code suspecte
- **Utilisez les fonctions "Watch" ou "Evaluate"** pour surveiller des variables spécifiques ou "évaluer" des expressions pendant le débogage. Utilisez-les pour voir comment une variable ou une expression évolue.
- **N'oubliez pas les fonctions natives** comme **pdb**
- **Apprenez les raccourcis clavier** pour accélérer le processus de débogage
- **Ne laissez pas les points d'arrêt obsolètes traîner** : supprimez les
- **Consultez la documentation** si vous êtes bloqué sur une erreur particulière

Conclusion



- Le débogage est **essentiel** pour détecter, comprendre et résoudre les erreurs
- Utilité des points d'arrêt pour suspendre l'exécution du code afin **d'examiner** le comportement et l'état du programme
- Exécution **pas à pas** pour avancer ligne par ligne dans le code
- Utilisez le débogueur, **même sans bug** : il peut être un outil d'apprentissage, si vous n'êtes pas sûr de la manière dont un morceau de code fonctionne, utilisez le débogueur pour le découvrir !

Chaque bug résolu est une **opportunité d'apprentissage** !



À votre tour

TD debugger

Objectif : À la fin de ce TD, vous serez en mesure d'utiliser de manière efficace et avancée le debugger de Visual Studio Code pour déboguer des scripts Python.

Exercice 1 : Installation et configuration initiale

1. Créez-en un nouveau projet td-1
2. Dans le menu latéral, cliquez sur l'icône "Run and Debug"

TD debugger

Exercice 2 : Points d'arrêt (breakpoints)

Créer un fichier **somme.py** avec le code suivant :

```
def somme(a, b):  
    result = a + b  
    return result  
  
for i in range(5):  
    print(somme(i, i*2))
```

1. Insérez un point d'arrêt (breakpoint) à la ligne `result = a + b`.
2. Démarrez le débogage. Observez comment le code s'arrête à ce point d'arrêt.
3. À l'aide de la fenêtre "VARIABLES", examinez les valeurs de `a`, `b` et `result`.

TD debugger

Exercice 3 : Stepping Through

Utilisez le même code que dans l'exercice précédent.

1. Lorsque vous atteignez le point d'arrêt, utilisez le bouton "Step Over" pour avancer ligne par ligne.
2. Essayez "Step Into" lorsque vous atteignez l'appel de la fonction `somme()`. Observez la différence avec "Step Over".
3. Utilisez "Step Out" pour sortir de la fonction.

Exercice 4 : Watch

1. Dans la fenêtre "WATCH", ajoutez les expressions `a`, `b` et `a * b`.
2. Relancez le débogage. Observez comment les valeurs de ces expressions changent à mesure que le code s'exécute.

TD debugger

Exercice 5 : Points d'arrêt conditionnels

1. Supprimez le point d'arrêt précédemment ajouté.
2. Ajoutez un nouveau point d'arrêt conditionnel à la ligne `print(somme(i, i*2))` avec la condition `i == 3`.
3. Démarrez le débogage. Observez comment le point d'arrêt ne s'active que lorsque `i` est égal à 3.

Exercice 6 : Exception Breakpoints

Créer un fichier **division.py** avec le code suivant :

```
def diviser(a, b):  
    return a / b  
  
print(diviser(10, 2))  
print(diviser(10, 0))
```

1. Dans la fenêtre "BREAKPOINTS", activez "Raised Exceptions".
2. Démarrez le débogage. Observez comment le debugger s'arrête lorsqu'une exception est levée.